WEATHER APPLICATION

React.js Web Application

Prepared By: Ritesh Chaudhary

Contents

1. Introduction **2**

2. Project Objectives **2**

3. Technologies and Tools Used **2**

4. System Design and Implementation **2**

4.1 Main Components 3

App.jsx 3

Input.jsx 3

Display.jsx 3

TempTimeDetail.jsx 3

Forecast.jsx 3

ForecastAccordianButton.jsx 3

4.2 Weather Service 3

5. Application Workflow **4**

6. API Integration and Data Processing **4**

7. Application Demonstration **5**

7.1 Desktop View 5

7.2 Mobile View 6

8. Testing and Observations **7**

9. Limitations and Future Improvements **7**

10. Conclusion **7**

11. References **8**

1. Introduction

The Weather Application is a web-based application developed using React.js and Vite. It retrieves real-time weather information and forecasts from the OpenWeather API and presents the information through a simple and interactive user interface.

Users can search for weather conditions by city name or use their current location to retrieve local weather information. The application also allows users to switch between Celsius and Fahrenheit and view additional weather details, including humidity, wind speed, sunrise, sunset, and upcoming forecasts.

The project demonstrates practical implementation of React components, state management, API integration, asynchronous JavaScript, and data transformation.

2. Project Objectives

The main objectives of this project are:

- To develop an interactive weather application using React.js.
- To integrate a third-party REST API for retrieving weather information.
- To implement city-based weather search and browser geolocation.
- To provide temperature unit selection between Celsius and Fahrenheit.
- To display current weather conditions and upcoming forecasts.
- To organize application logic using reusable components and a separate service layer.

3. Technologies and Tools Used

Technology	**Purpose**
React.js	Building the user interface using reusable components
Vite	Development server and production build tool
JavaScript (ES6+)	Application logic and data processing
OpenWeather API	Retrieving current weather and forecast information
Luxon	Formatting dates and local times
React Icons	Providing weather and interface icons
CSS	Designing and styling the application
VS Code	Development environment

4. System Design and Implementation

The application follows a component-based architecture. The user interface and API-related operations are separated to make the application easier to understand, maintain, and extend.

4.1 Main Components

App.jsx

Acts as the main application component. It manages the application state using React's *useState* and retrieves weather information using *useEffect*. It also passes weather data and user selections to the child components through props.

Input.jsx

Provides the city search functionality, current-location access, and temperature unit selection. It uses React state to manage the search input and the browser Geolocation API to retrieve the user's coordinates.

Display.jsx

Displays the main weather information, including city name, country, current temperature, local time, and date. The temperature symbol changes dynamically according to the selected unit.

TempTimeDetail.jsx

Displays additional weather information, including feels-like temperature, humidity, wind speed, sunrise, sunset, and minimum and maximum temperatures.

Forecast.jsx

Displays hourly and daily weather forecasts. The same component is reused with different forecast data, reducing code duplication.

ForecastAccordianButton.jsx

Allows users to show or hide the forecast section.

4.2 Weather Service

The *weatherService.*`js` file contains the application's API communication and data-formatting logic.

It is responsible for:

- Constructing API requests using the OpenWeather API.
- Retrieving current weather information.
- Retrieving forecast information using latitude and longitude.
- Extracting the required information from API responses.
- Formatting weather timestamps using Luxon.

- Generating weather icon URLs.
- Returning formatted weather data to the React application.

Separating these operations from the UI components makes the application structure easier to maintain and understand.

5. Application Workflow

The application follows the workflow below:

1. The application initially loads weather information for Kathmandu.
2. The user can search for another city or select the current-location option.
3. React updates the search query or location coordinates.
4. The *useEffect* hook detects changes in the search query or temperature unit.
5. The weather service requests current weather information from the OpenWeather API.
6. The service uses the returned latitude and longitude to request forecast information.
7. The retrieved API responses are formatted into the required data structure.
8. React updates the weather state and displays the information through the respective components.

6. API Integration and Data Processing

The application integrates the OpenWeather API through two endpoints:

- **Current Weather API:** Retrieves current weather conditions for a specified city or geographical coordinates.
- **5-Day Forecast API:** Retrieves forecast information at three-hour intervals.

The application uses the *fetch()* method with *async/await* to communicate with the API.

The API responses contain various weather attributes. The service extracts only the required information and organizes it into a simplified JavaScript object before passing it to the React components.

This approach avoids unnecessary API response data being passed throughout the application.

The application also uses Luxon to convert Unix timestamps into readable local dates and times using the location's timezone information.

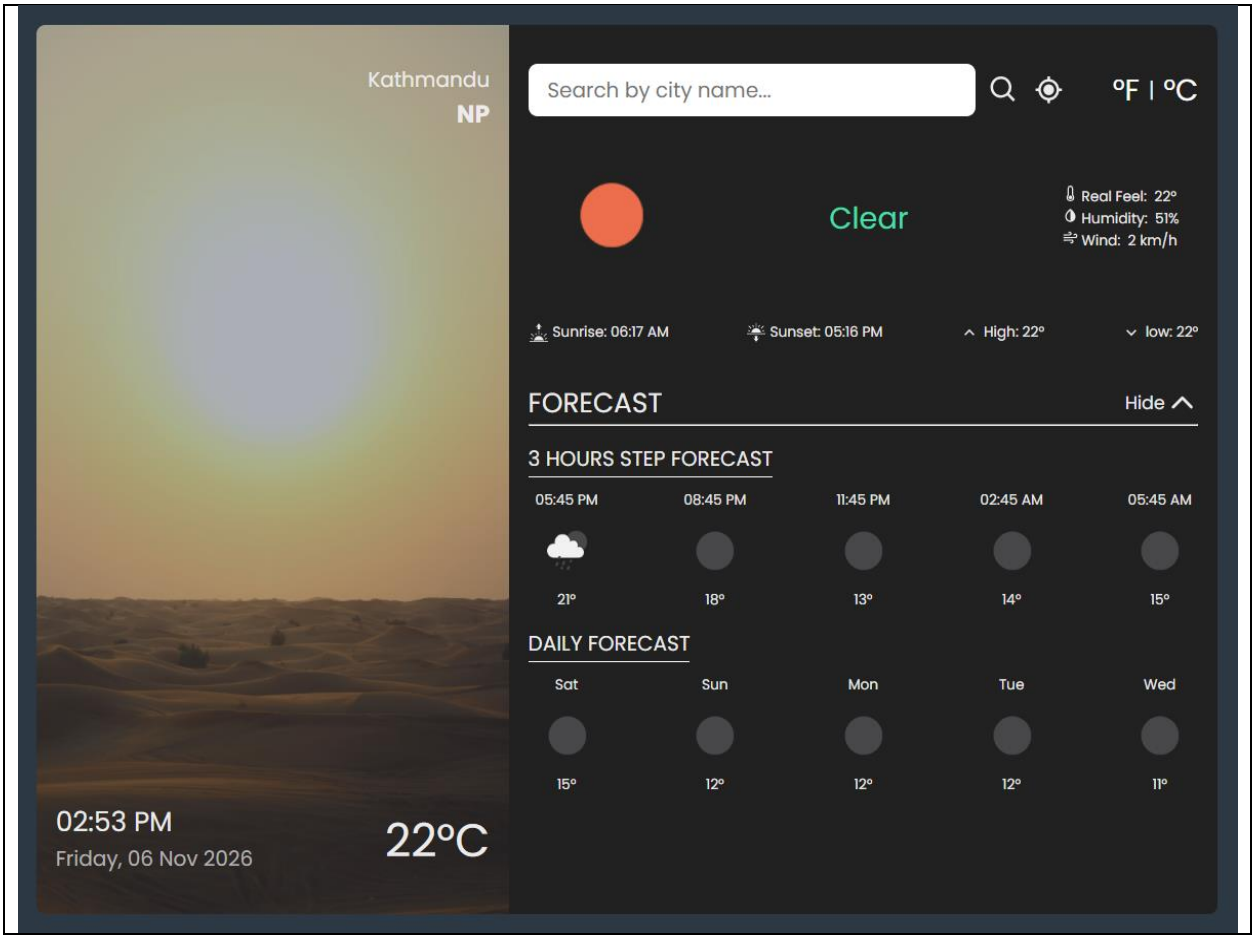
7. Application Demonstration

The following screenshots demonstrate the application's user interface on different screen sizes.

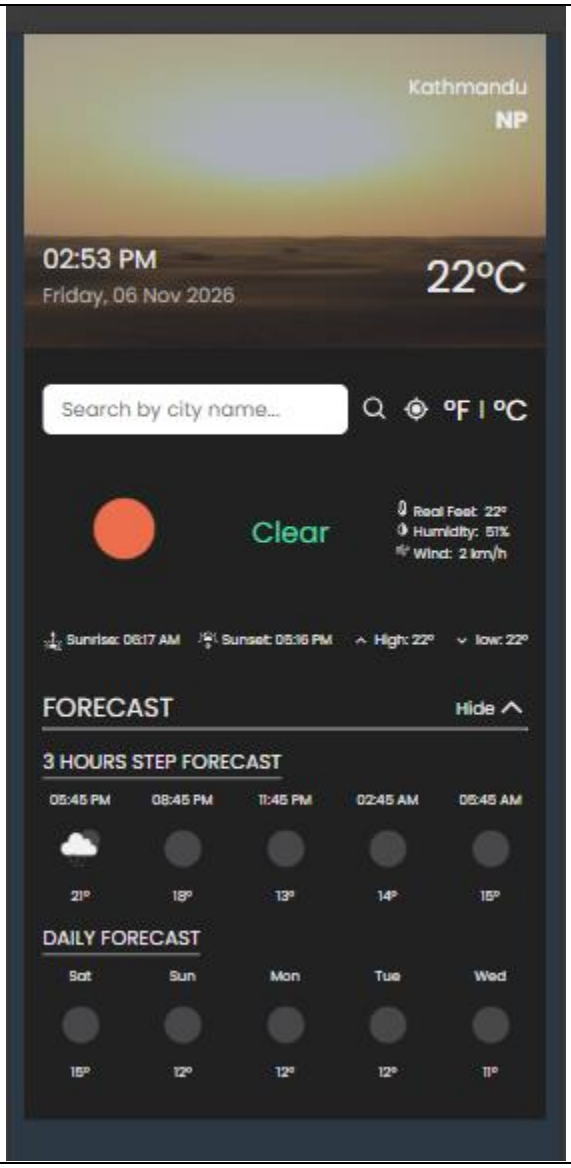
Live Application: https://weather-app-3qm6.onrender.com/

Source Code: https://github.com/riteshchaudhary375/weather-app?tab=readme-ov-file

7.1 Desktop View



7.2 Mobile View



The interface provides city search, temperature selection, current weather information, additional weather details, and forecast information in an organized layout.

8. Testing and Observations

The application was designed around the following functional test scenarios:

Test Scenario	Expected Result
Initial application load	Kathmandu weather information is requested
Search by city name	Weather information for the searched city is displayed
Current location	Weather information is requested using geographical coordinates
Select Celsius	Weather information is displayed in Celsius
Select Fahrenheit	Weather information is displayed in Fahrenheit
Display forecast	Hourly and daily forecasts are displayed
Hide forecast	Forecast information is hidden
Invalid city name	API error is handled by the service and application

The application demonstrates the expected functionality of its main weather-search, temperature-selection, and forecast features. Additional user-facing error and loading indicators can improve the overall experience.

9. Limitations and Future Improvements

Although the application provides its core weather functionalities, several improvements could be introduced in future versions:

- Implement loading indicators while retrieving weather information.
- Display user-friendly error messages for invalid searches and API failures.
- Add weather-based background images.
- Implement recent searches and favorite cities.
- Include additional information such as air quality and UV index.
- Improve accessibility and responsive interface behavior.
- Introduce a backend API layer to protect API credentials in production.

10. Conclusion

The Weather Application demonstrates the practical use of React.js for developing an interactive web application integrated with a third-party REST API.

Through this project, I implemented reusable React components, state management, asynchronous API requests, browser geolocation, and weather data transformation. Separating

API communication and data formatting into a dedicated service file also helped maintain a clear distinction between application logic and user interface components.

Overall, the project strengthened my understanding of React development, API integration, JavaScript programming, and component-based application architecture.

11. References

1. OpenWeather – https://openweathermap.org/api
2. React Documentation – https://react.dev/
3. Vite Documentation – https://vite.dev/
4. Luxon Documentation – https://moment.github.io/luxon/